

UCC 122-1

Cloud Architecture

Lab 4: Designing a microservices architecture for Digi Bank with Spring Boot

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

June 2026

Part I: Implementation of the Microservices Architecture

Context

In workshop 3, students built a first usable version of Digi Bank as a modular monolith using Spring Boot, PostgreSQL, Maven, a WAR package, and deployment on WildFly. This achievement allowed them to stabilize the business scope of the core project, validate the main entities, expose REST endpoints, implement unit tests, and prepare a coherent technical foundation for future architectural evolution.

This workshop proposes a transition to a microservices architecture. The goal is no longer to group all functions into a single application deployed as a single unit, but to distribute business responsibilities across several autonomous services, each responsible for a specific area of the banking system. This evolution addresses concrete needs for availability, independent deployment, resilience, scalability, and more granular maintenance.

Digi Bank retains the same core functionality: customer management, account management, transaction management, compliance monitoring, notifications, and application entry point. However, execution now relies on a constellation of distinct Spring Boot services, orchestrated with Docker, managed by GitHub Actions, and structured according to the microservices principles covered in the course. The project remains intentionally educational: it demonstrates the fundamentals of a distributed architecture without veering into excessive industrial complexity.

Scenario

Digi Bank's technical team wants to evolve the platform towards a more flexible and modern architecture. The modular monolith implemented in previous workshops validated the domain, but it shows its limitations when it comes to independently deploying certain functions, isolating faults, or upgrading a module without redeploying the entire system.



In this new context, developers must break down the application into separate services, expose consistent REST APIs, organize inter-service communication, manage the persistence of each service, and prepare a reproducible containerized deployment. Your role is to produce a first stable, readable, and directly usable technical version of Digi Bank as a microservice, with a documented, phased implementation strategy.

Workshop objectives

At the end of this workshop, the student should be able to:

- **A4.1:** Analyze a business need and deduce coherent service boundaries.
- **A4.2:** Design a microservices architecture adapted to Digi Bank
- **A4.3:** Create multiple standalone Spring Boot projects
- **A4.4:** Configure PostgreSQL for a service-based database model
- **A4.5:** Implement REST endpoints in each service
- **A4.6:** Implement interdepartmental calls
- **A4.7:** Introduce an API Gateway as a single entry point
- **A4.8:** Containerizing services with Docker
- **A4.9:** Automating build and deployment with GitHub Actions
- **A4.10:** Apply the relevant microservices patterns to the Digi Bank case.

Resources provided

The students have the following materials to complete the workshop:

- The Digi Bank overarching project and its functional framework;
- Workshop 2, which laid the foundations for the modular Jakarta EE monolith;
- Workshop 3, which provided a consistent Spring Boot implementation;
- The course on cloud architecture, monolithic architectures and modular structuring;
- The fundamental principles of microservices, containerization, and CI/CD automation.



Technical prerequisites

The technical prerequisites are as follows:

- JDK 17 or compatible version
- Apache Maven
- PostgreSQL
- Docker and Docker Compose
- Git
- A GitHub / GitLab account
- IntelliJ IDEA or Visual Studio Code
- A machine powerful enough to run multiple services in parallel.

Workshop pedagogy

The workshop follows a progressive structure. It begins with environment preparation, then defines the functional breakdown, before creating the core services, adding orchestration, and finally preparing for containerization and automation. This progression allows students to understand that microservices architecture is not just a technical choice, but a way to organize the system around independent business responsibilities.

In the case of Digi Bank, this approach is particularly interesting because it allows us to illustrate the real tensions of a distributed banking system: coordination of operations, data consistency, fault tolerance, isolation of critical functionalities and supervision of exchanges.

Target architecture

The target architecture is based on the following components:

- **API gateway:** single entry point for the application
- **customer service:** customer management
- **account-service:** account management
- **Transaction service:** financial operations management
- **compliance service:** compliance rules
- **notification service:** business or technical notifications
- **discovery-server:** service discovery
- **config-server :** externalized configuration
- **a PostgreSQL database per service**
- **Docker Compose orchestration**
- **a GitHub Actions pipeline** for building and validation.

The logic of the system remains simple to read: the client calls the gateway, the gateway forwards the request to the correct service, each service communicates with its own database, and critical services communicate with each other via explicit calls or appropriate coordination mechanisms.

Functional view

Digi Bank should allow us to illustrate the following functionalities:

- Customer creation and consultation;
- Account creation and consultation;
- Execution and consultation of transactions;
- Compliance validation prior to processing;
- Sending notifications after sensitive operations;
- Unified access via the Gateway API;
- Supervision of distributed behavior across the studied patterns.

1. Environmental preparation

Before developing Digi Bank using microservices, the student must have a complete, consistent working environment ready to run multiple applications in parallel. The goal of this phase is to avoid technical roadblocks when the services are launched separately, connected to each other, and tested in Docker.

1.1 Installation of tools

Install or check the following components:

- Java JDK 17
- Apache Maven
- PostgreSQL
- Docker Desktop or Docker Engine
- Docker Compose
- Git
- IntelliJ IDEA Community/Ultimate or Visual Studio Code
- A working GitHub/Gitlab account
- Postman for testing APIs.

These tools form the technical basis of the workshop. Java and Maven are used to build the services, PostgreSQL ensures persistence, Docker enables reproducible execution, and GitHub Actions will prepare the automation of the delivery chain.

1.2 Basic Checks

The student must verify the installation with the following commands:

```
Java -version
mvn -version
git --version
psql --version
docker --version
```

```
docker compound version
```

The results should confirm that the tools are correctly detected by the system and usable without additional configuration. If a command fails, the environment must be corrected before development begins.

```
user@ca:4f:12:a0:b3:c1 ~ % Java -version

mvn -version

git --version

psql --version

docker --version

docker compound version


Java version "17.0.16" 2025-07-15 LTS

Java(TM) SE Runtime Environment (build 17.0.16+12-LTS-247)

Java HotSpot (TM) 64-Bit Server VM (build 17.0.16+12-LTS-247, mixed fashion,
(sharing))

Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)

Maven home: /opt/homebrew/Cellar/maven/3.9.11/libexec

Java version: 17.0.16, vendor: Oracle Corporation, runtime:
/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home

Default local: fr_FR, platform encoding: UTF-8

BONE name: "mac os x", version: "26.5.1", arch: "aarch64", family: "mac"

git version 2.50.1 (Apple Git-155)

psql (PostgreSQL) 18.4 (Homebrew)

Docker version 28.5.1, build e180ab8

Docker Compound version v2.40.3-desktop.1
```

```
user@ca:4f:12:a0:b3:c1 ~ %
```

1.3 Expected Result

The local environment must be capable of:

- Compiling a Spring Boot project
- Run multiple services simultaneously
- Starting PostgreSQL databases
- Building and launching Docker containers
- Testing REST endpoints locally.

2. IDE configuration

The development environment must be prepared in the chosen IDE so that the services are correctly recognized by the tool, the Maven dependencies are automatically loaded, and the tests are immediately visible.

2.1 Configuration under IntelliJ IDEA

The recommended steps are as follows:

1. Open IntelliJ IDEA.
2. Create a new Maven project or import the existing repository.
3. Define the Java SDK 17.
4. Enable automatic import of Maven projects.
5. Verify the recognition of `src/main/java` and `src/test/java`.
6. Install or activate useful plugins for Spring, Maven, JUnit and Git.

2.2 Configuration under Visual Studio Code

Install the following extensions:

- Extension Pack for Java

- Maven for Java
- Language Support for Java by Red Hat
- Debugger for Java
- Test Runner for Java
- Spring Boot Extension Pack
- Cucumber Gherkin support.

It is also necessary to verify that JDK 17 is correctly referenced and that the Maven and YAML files are recognized without errors. This configuration is important because the workshop will involve several modules, services, and orchestration files.

2.3 Expected result

The IDE must enable:

- Navigate easily through each service;
- Launch a service individually;
- Run the unit tests;
- Edit configuration files unambiguously;
- Quickly identify dependency or compilation errors.

3. Domain division

The first conceptual task is to transform the banking sector into independent services. In Digi Bank, the separation of components is avoided by mere technical chance: each service must correspond to an identifiable business capability.

3.1 Services retained

The chosen breakdown is as follows:

- **customer service:** customer management
- **account-service:** account management

- **transaction-service:** transaction management
- **compliance service:** compliance control
- **notification-service:** notifications
- **API Gateway:** Single point of entry
- **discovery-server:** discovery registry
- **config-server:** centralization of settings.

3.2 Business logic of each service

- The **customer service department** manages identity and contact information.
- The **account-service** handles bank accounts and their status.
- The **transaction service** applies the financial operations and orchestrates the necessary validations.
- The **compliance service** verifies regulatory or internal rules.
- The **notification service** distributes messages related to validated or rejected operations.

3.3 Expected Result

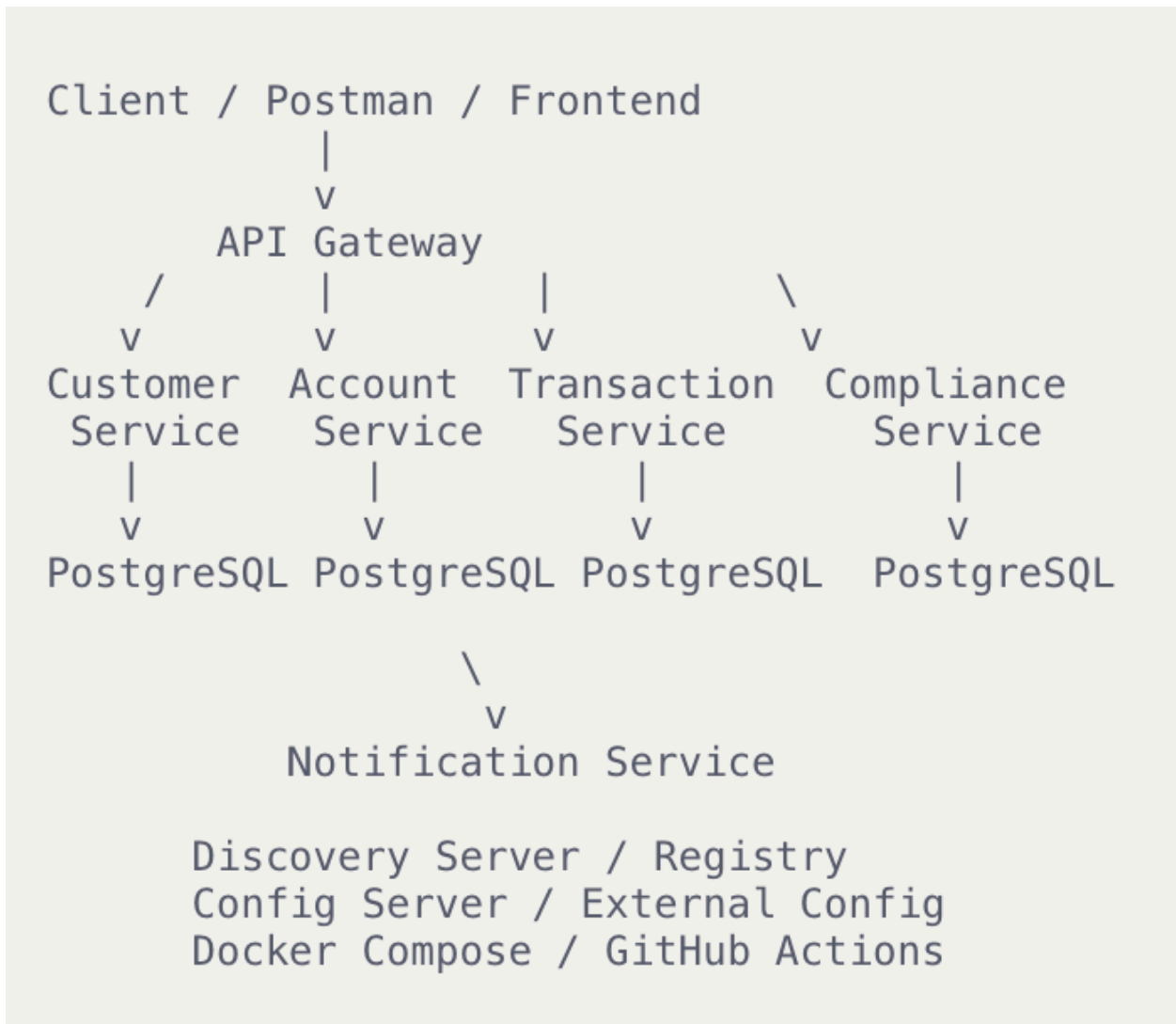
The student must be able to explain that this breakdown allows:

- to isolate responsibilities
- to evolve a service without breaking the entire system
- to deploy the bricks independently
- to better control breakdowns
- to simplify targeted testing.

4. Detailed architectural diagram

The architecture diagram must clearly show the relationships between the components. In a practical exercise, a readable diagram is as important as the code itself, as it helps to understand the path of a request and the responsibility of each component.

4.1 Overview



4.2 Reading the diagram

The client never accesses internal services directly. Instead, they use the API Gateway, which handles routing and cross-functional processing. Services use their own databases, avoiding tight coupling. The discovery server identifies services by logical name, and Docker Compose orchestrates everything in a reproducible local environment.

4.3 Educational Expectations

The student must understand that this architecture introduces new problems compared to the monolithic architecture: distributed coordination, partial failures, fragmented configuration, variable availability, and the need for observability. It is precisely to address these problems that microservices patterns will be studied in the remainder of the workshop.

5. Creation of the project structure

5.1 Organizational choices

For the workshop, we've chosen a single repository containing all the services. This organization simplifies teaching, corrections, and the progressive demonstration of patterns. It allows us to centralize orchestration files, scripts, and common configurations.

5.2 General Tree Structure

The following tree structure is recommended:

```
digibank-microservices/  
  api-gateway/  
  discovery-server/  
  config-server/  
  customer-service/  
  account-service/  
  transaction-service/  
  compliance-service/  
  notification-service/  
  docker/  
  .github/workflows/  
  docker-compose.yml  
  README.md
```

This structure makes the separation of responsibilities visible while keeping the whole thing readable for a student.

5.3 Internal structure of a service

Each Spring Boot service must follow a simple and regular internal organization:

```
src/main/java/com/digibank/<service>/
  api/
  service/
  repository/
  model/
  dto/
  config/

src/main/resources/
  application.yml
  application-dev.yml
  application-prod.yml

src/test/java/
  unit/
  integration/
```

This homogeneity facilitates maintenance, corrections and cross-reading between services.

6. Basic Maven Structure

Each service will be a standalone Maven project. This decision allows each service to be compiled, tested, and deployed separately, which aligns with the microservices philosophy.

6.1 Parent pom.xml file

A common Maven parent can be used to harmonize versions:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" >

<modelVersion> 4.0.0 </modelVersion>

<parent>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-parent </artifactId>

<version> 3.4.0 </version>

<relativePath/>

</parent>

<groupId> com.digibank </groupId>

<artifactId> digibank-microservices-parent </artifactId>

<version> 1.0.0 </version>

<packaging> pom </packaging>

<modules>

<module> api-gateway </module>

<module> discovery-server </module>

<module> config-server </module>

<module> customer-service </module>

<module> account-service </module>

<module> transaction-service </module>

<module> compliance-service </module>

<module> notification-service </module>
```

```

</modules>

<properties>
<java.version> 17 </java.version>

<spring.boot.version> 3.4.0 </spring.boot.version>

<maven.compiler.release> 17 </maven.compiler.release>

<project.build.sourceEncoding> UTF-8 </project.build.sourceEncoding>

<junit.version> 5.10.2 </junit.version>

<cucumber.version> 7.15.0 </cucumber.version>

<junit.platform.version> 1.12.2 </junit.platform.version>

<spring-cloud.version> 2024.0.1 </spring-cloud.version>

</properties>

</project>

```

This parent does not impose the business logic, but it centralizes the versions, which simplifies the evolution of the entire project.

6.2 Pom of a typical service

Each business service can declare a classic Spring Boot structure:

```

<project xmlns="http://maven.apache.org/POM/4.0.0" >

<modelVersion> 4.0.0 </modelVersion>

<parent>

```



```
<groupId> com.digibank </groupId>

<artifactId> digibank-microservices-parent </artifactId>

<version> 1.0.0 </version>

</parent>

<artifactId> customer-service </artifactId>

<dependencies>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-web </artifactId>

</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-data-jpa </artifactId>

</dependency>

<dependency>

<groupId> org.postgresql </groupId>

<artifactId> postgresql </artifactId>

<scope> runtime </scope>

</dependency>

<dependency>

<groupId> org.junit.jupiter </groupId>

<artifactId> junit-jupiter </artifactId>
```

```
<version> ${junit.version} </version>

<scope> test </scope>

</dependency>

<dependency>

<groupId> org.junit.platform </groupId>

<artifactId> junit-platform-suite-api </artifactId>

<version> ${junit.platform.version} </version>

<scope> test </scope>

</dependency>

<dependency>

<groupId> org.junit.platform </groupId>

<artifactId> junit-platform-suite </artifactId>

<version> ${junit.platform.version} </version>

<scope> test </scope>

</dependency>

<dependency>

<groupId> io.cucumber </groupId>

<artifactId> cucumber-java </artifactId>

<version> ${cucumber.version} </version>

<scope> test </scope>

</dependency>

<dependency>

<groupId> io.cucumber </groupId>

<artifactId> cucumber-junit-platform-engine </artifactId>
```

```
<version> ${cucumber.version} </version>

<scope> test </scope>

</dependency>

</dependencies>

</project>
```

Each service can then add its specific dependencies without disrupting the others.

6.3 Expected Result

The student must be able to create an independent Spring Boot service, compile it with Maven and launch it without depending on another application module for its local execution.

7. Basic service configuration

7.1 Common Properties

Each service must have its own application.yml file. A minimal configuration might look like this:

```
server :

  port : 8081

spring :
```

```
application :  
    name : customer service  
datasource :  
    url : jdbc:postgresql://localhost:5432/digibank_customer_db  
    username : digibankuser  
    password : digibankpwd  
    driver-class-name : org.postgresql.Driver  
jpa :  
    hibernate :  
        ddl-auto : update  
    properties :  
        hibernate :  
            dialect : org.hibernate.dialect.PostgreSQLDialect
```

7.2 Creating PostgreSQL databases

We can plan for a base per service:

```
CREATE DATABASE digibank_customer_db ;  
CREATE DATABASE digibank_account_db ;  
CREATE DATABASE digibank_transaction_db ;  
CREATE DATABASE digibank_compliance_db ;  
CREATE DATABASE digibank_notification_db ;
```



This separation strengthens the autonomy of the services and avoids the direct sharing of a single scheme.

7.3 Expected Result

Each service must be able to start with its own dedicated database and expose its endpoints independently of the others.

8. First business implementation

8.1 Customer Service

The customer service must allow the creation and viewing of a customer. It includes a Customer entity, a repository, a business service, and a REST controller.

8.2 Service Account

The account-service manages the accounts and must be able to associate an account with a customer.

8.3 Transaction Service

The transaction service records financial operations and interacts with the compliance service before validation.

8.4 Service Compliance

The compliance department checks whether an amount is authorized according to the establishment's rules.

8.5 Service Notification

The notification service simulates sending a message after certain business operations.

8.6 Expected Result

At this stage, the student needs to understand how the departments share responsibilities and how the first interdepartmental exchanges will be set up in the following parts of the workshop.

9. Customer Service

Customer **service** manages Digi Bank's clients. Without a dedicated service, identity data would be mixed with account and transaction information, making system evolution more difficult and the business structure less clear. This service must allow users to create, view, modify, and list clients.

7.1 Service Structure

The recommended internal directory structure is as follows:

```
customer-service/  
  src/main/java/com/digibank/customer/  
    CustomerServiceApplication.java  
    api/CustomerController.java  
    model/Customer.java  
    repository/CustomerRepository.java  
    service/CustomerService.java  
  src/main/resources/  
    application.yml
```

7.2 Main Class

```
package com.digibank.customer;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class CustomerServiceApplication {  
    public static void main (String[] args) {  
        SpringApplication.run(CustomerServiceApplication.class, args);  
    }  
}
```

7.3 Customer Entity

```
package com.digibank.customer.model;

import jakarta.persistence.*;

@Entity
@Table (name = "customers" )
public class Customer {

    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)
    private Long id ;

    @Column (nullable = false )
    private String firstName ;

    @Column (nullable = false )
    private String lastName ;

    @Column (nullable = false , unique = true )
    private String email ;

    public Customer () {
    }
}
```



```
public Customer (String firstName, String lastName, String
email) {

    this.firstName = firstName ;

    this.lastName = lastName ;

    this.email = email ;

}

public Long getId () {

    return id ;

}

public String getFirstName () {

    return firstName ;

}

public void setFirstName (String firstName) {

    this.firstName = firstName ;

}

public String getLastName () {

    return lastName ;

}
```

```
public void setLastName (String lastName) {

    this.lastName = lastName ;

}

public String getEmail () {

    return email ;

}

public void setEmail (String email) {

    this.email = email ;

}

}
```

7.4 Repository CustomerRepository

```
package com.digibank.customer.repository;

import com.digibank.customer.model.Customer;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends
JpaRepository<Customer, Long> {
```

```
        boolean existsByEmail (String email);  
    }  
}
```

7.5 Business Service Customer Service

```
package com.digibank.customer.service;  
  
import com.digibank.customer.model.Customer;  
import com.digibank.customer.repository.CustomerRepository;  
import org.springframework.stereotype.Service;  
  
import java.util.List;  
import java.util.Optional;  
  
@Service  
public class CustomerService {  
  
    private final CustomerRepository repository ;  
  
    public CustomerService (CustomerRepository repository) {  
        this . repository = repository;  
    }  
}
```

```

public Customer createCustomer (Customer customer) {

    if ( repository .existsByEmail(customer.getEmail())) {

        throw new IllegalArgumentException( "Email already
exists" );

    }

    return repository .save(customer);

}

public Optional<Customer> getCustomer (Long id) {

    return repository .findById(id);

}

public List<Customer> getAllCustomers () {

    return repository.findAll ();

}

public Customer updatedCustomer (Long id, Customer
updatedCustomer) {

Customer existing = repository .findById(id)

.orElseThrow(() -> new IllegalArgumentException( "Customer not
found" ));

existing.setFirstName(updatedCustomer.getFirstName());

existing.setLastName(updatedCustomer.getLastName());

existing.setEmail(updatedCustomer.getEmail());

```

```
        return repository .save(existing) ;  
    }  
}
```

7.6 REST Controller CustomerController

```
package com.digibank.customer.api;  
  
import com.digibank.customer.model.Customer;  
import com.digibank.customer.service.CustomerService;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping ( "/api/customers" )  
public class CustomerController {  
  
    private final CustomerService service ;  
  
    public CustomerController (CustomerService service) {  
        this.service = service ;  
    }  
}
```

```

    @PostMapping

    public ResponseEntity<Customer> createCustomer ( @RequestBody
Customer customer) {

        return ResponseEntity.status(HttpStatus.CREATED).body(
service .createCustomer(customer));
    }


    @GetMapping (("/{id}") )

    public ResponseEntity<Customer> findById ( @PathVariable Long
id) {

        return service .getCustomer(id)

.map(ResponseEntity::ok)

.orElse(ResponseEntity.notFound().build());
    }


    @GetMapping

    public ResponseEntity<?> findAll () {

        return ResponseEntity.ok( service.getAllCustomers ());
    }


    @PutMapping (("/{id}") )

    public ResponseEntity<Customer> updateCustomer (
@PathVariable Long id, @RequestBody Customer customer) {

        return ResponseEntity.ok( service.updateCustomer (id,
customer));
    }

```

```
}  
}
```

9.7 Configuring application.yml

```
server :  
  
  port : 8081  
  
spring :  
  
  application :  
  
    name : customer service  
  
  datasource :  
  
    url : jdbc:postgresql://localhost:5432/digibank_customer_db  
  
    username : digibankuser  
  
    password : digibankpwd  
  
    driver-class-name : org.postgresql.Driver  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : update  
  
    properties :  
  
      hibernate :  
  
        dialect : org.hibernate.dialect.PostgreSQLDialect
```

10. Service Account

The **account service** represents bank accounts. Without this service, financial operations would be too closely tied to the customer database, and it would be difficult to independently change account management rules.

10.1 Service Structure

```
account-service/  
  src/main/java/com/digibank/account/  
    AccountServiceApplication.java  
    api/AccountController.java  
    model/Account.java  
    repository/AccountRepository.java  
    service/AccountService.java  
  src/main/resources/  
    application.yml
```

10.2 Main Class

```
package com.digibank.account;  
  
import org.springframework.boot.SpringApplication;  
  
import  
org.springframework.boot.autoconfigure.SpringBootApplication;
```



```

@SpringBootApplication

public class AccountServiceApplication {

    public static void main (String[] args) {

SpringApplication.run(AccountServiceApplication.class , args);

    }

}

```

10.3 Entity Account

```

package com.digibank.account.model;

import jakarta.persistence.*;

@Entity
@Table (name = "accounts" )

public class Account {

    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)

    private Long id ;

    @Column (nullable = false , unique = true )

    private String accountNumber ;

```

```
@Column (nullable = false )  
  
private Double balance ;  
  
@Column (nullable = false )  
  
private Long customerId ;  
  
public Account () {  
}  
  
public Account (String accountNumber, Double balance, Long  
customerId) {  
  
    this . accountNumber = accountNumber;  
  
    this.balance = balance ;  
  
    this . customerId = customerId;  
}  
  
public Long getId () {  
  
    return id ;  
}  
  
public String getAccountNumber () {  
  
    return accountNumber ;  
}
```

```
public void setAccountNumber (String accountNumber) {  
    this . accountNumber = accountNumber;  
}  
  
public Double getBalance () {  
    return balance ;  
}  
  
public void setBalance (Double balance) {  
    this.balance = balance ;  
}  
  
public Long getCustomerId () {  
    return customerId ;  
}  
  
public void setCustomerId (Long customerId) {  
    this . customerId = customerId;  
}  
}
```

10.4 Repository AccountRepository

```
package com.digibank.account.repository;

import com.digibank.account.model.Account;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountRepository extends
JpaRepository<Account, Long> {

    boolean existsByAccountNumber (String accountNumber);
}
```

10.5 Business Service AccountService

```
package com.digibank.account.service;

import com.digibank.account.model.Account;
import com.digibank.account.repository.AccountRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.UUID;

@Service
```

```
public class AccountService {

    private final AccountRepository repository ;

    public AccountService (AccountRepository repository) {

        this . repository = repository;
    }

    public Account createAccount (Account account) {

        if (account.getAccountNumber() == null ||
account.getAccountNumber().isBlank()) {
account.setAccountNumber( "ACC-" + UUID.randomUUID ( ));
        }

        if (account.getBalance() == null ) {
account.setBalance( 0.0 );
        }

        return repository .save(account);
    }

    public List<Account> getAccounts () {

        return repository.findAll ();
    }

    public Account getAccount (Long id) {
```

```
        return repository .findById(id)

        .orElseThrow(() -> new IllegalArgumentException( "Account not
found" ));
    }
}
```

10.6 REST Controller AccountController

```
package com.digibank.account.api;

import com.digibank.account.model.Account;
import com.digibank.account.service.AccountService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping ( "/api/accounts" )
public class AccountController {

    private final AccountService service ;

    public AccountController (AccountService service) {
```

```

        this.service = service ;
    }

    @PostMapping
    public ResponseEntity<Account> createAccount ( @RequestBody
Account account) {

        return ResponseEntity.status (HttpStatus.CREATED) .body (
service .createAccount (account)) ;
    }

    @GetMapping
    public ResponseEntity<?> findAll () {

        return ResponseEntity.ok( service.getAccounts ());
    }

    @GetMapping (("/{id}") )
    public ResponseEntity<Account> findById ( @PathVariable Long
id) {

        return ResponseEntity.ok( service.getAccount (id));
    }
}

```

10.7 Configuring application.yml

```
server :

port : 8082


spring :

  application :

    name : account-service

  datasource :

    url : jdbc:postgresql://localhost:5432/digibank_account_db

    username : digibankuser

    password : digibankpwd

    driver-class-name : org.postgresql.Driver

  jpa :

    hibernate :

      ddl-auto : update

    properties :

      hibernate :

        dialect : org.hibernate.dialect.PostgreSQLDialect
```

11. Transaction Service

The **transaction service** is the operational core of Digi Bank. Without a dedicated service, the processing of financial transactions would be fragmented and difficult to

secure. It is also the service that will communicate most frequently with the other components.

11.1 Service Structure

```
transaction-service/  
  src/main/java/com/digibank/transaction/  
    TransactionServiceApplication.java  
    api/TransactionController.java  
    model/Transaction.java  
    repository/TransactionRepository.java  
    service/TransactionService.java  
    client/ComplianceClient.java  
  src/main/resources/  
    application.yml
```

11.2 Main Class

```
package com.digibank.transaction;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class TransactionServiceApplication {  
    public static void main (String[] args) {  
        SpringApplication.run(TransactionServiceApplication.class, args);  
    }  
}
```

```
}  
  
}
```

11.3 Transaction Entity

```
package com.digibank.transaction.model;  
  
import jakarta.persistence.*;  
  
@Entity  
@Table (name = "transactions" )  
public class Transaction {  
  
    @Id  
    @GeneratedValue (strategy = GenerationType.IDENTITY)  
    private Long id ;  
  
    @Column (nullable = false )  
    private String type ;  
  
    @Column (nullable = false )  
    private Double amount ;  
  
    @Column (nullable = false )
```

```
private Long accountId ;

public Transaction () {

}

public Transaction (String type, Double amount, Long accountId) {

    this.type = type ;

    this.amount = amount ;

    this.accountId = accountId ;

}

public Long getId () {

    return id ;

}

public String getType () {

    return type ;

}

public void setType (String type) {

    this.type = type ;

}

public Double getAmount () {

    return amount ;

}
```

```

}

    public void setAmount (Double amount) {

        this.amount = amount ;

    }

    public Long getAccountId () {

        return accountId ;

    }

    public void setAccountId (Long accountId) {

        this.accountId = accountId ;

    }

}

```

11.4 Repository TransactionRepository

```

package com.digibank.transaction.repository;

import com.digibank.transaction.model.Transaction;
import org.springframework.data.jpa.repository.JpaRepository;

public interface TransactionRepository extends
JpaRepository<Transaction, Long> {

}

```

11.5 Compliance Client

```
package com.digibank.transaction.client;

import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

@Component
public class ComplianceClient {

    private final RestTemplate restTemplate ;

    public ComplianceClient (RestTemplate restTemplate) {

        this . restTemplate = restTemplate;
    }

    public boolean validateAmount (Double amount) {
Boolean result = restTemplate.getForObject (
        "http://localhost:8084/api/compliance/validate/" +
amount,
Boolean.class
    );

    return Boolean. TRUE .equals(result);
}
```

```
}
```

11.6 RestTemplateConfig configuration class

```
package com.digibank.transaction.configuration;

import org.springframework.context.annotation. Bean ;
import org.springframework.context.annotation. Setup ;
import org.springframework.web.client.RestTemplate;

@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate () {
        return new RestTemplate();
    }
}
```

11.7 Business Service TransactionService

```
package com.digibank.transaction.service;

import com.digibank.transaction.client.ComplianceClient;
```

```
import com.digibank.transaction.model.Transaction;

import com.digibank.transaction.repository.TransactionRepository;

import org.springframework.stereotype.Service;

import java.util.List;

@Service

public class TransactionService {

    private final TransactionRepository repository ;

    private final ComplianceClient complianceClient ;

    public TransactionService (TransactionRepository repository,
ComplianceClient complianceClient) {

        this . repository = repository;

        this . complianceClient = complianceClient;
    }

    public Transaction createTransaction (Transaction transaction) {

        if (! complianceClient
.validateAmount(transaction.getAmount())) {

            throw new IllegalArgumentException( "Transaction rejected
by compliance" );

        }

        return repository .save(transaction);
    }
}
```

```
public List<Transaction> getTransactions () {  
  
    return repository.findAll ();  
  
}  
}
```

11.8 REST Transaction Controller

```
package com.digibank.transaction.api;  
  
import com.digibank.transaction.model.Transaction;  
import com.digibank.transaction.service.TransactionService;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping ( "/api/transactions" )  
public class TransactionController {  
  
    private final TransactionService service ;  
  
    public TransactionController (TransactionService service) {  
        this.service = service ;  
    }  
}
```



```

    @PostMapping
    public ResponseEntity<Transaction> createTransaction (
@RequestBody Transaction transaction) {

        return ResponseEntity.status(HttpStatus.CREATED).body( service
.createTransaction(transaction));
    }

    @GetMapping
    public ResponseEntity<?> findAll () {

        return ResponseEntity.ok( service.getTransactions ());
    }
}

```

11.9 Configuring application.yml

```

server :

port : 8083


spring :

    application :

        name : transaction-service

    datasource :

        url : jdbc:postgresql://localhost:5432/digibank_transaction_db

        username : digibankuser

```

```
password : digibankpwd

driver-class-name : org.postgresql.Driver

jpa :

hibernate :

    ddl-auto : update

properties :

    hibernate :

        dialect : org.hibernate.dialect.PostgreSQLDialect
```

12. Compliance Department

The **compliance department** applies the business rules that govern the execution of sensitive operations. In a bank, this layer is essential because it allows unauthorized transactions to be blocked before they impact the rest of the system.

12.1 Service Structure

```
compliance-service/
  src/main/java/com/digibank/compliance/
    ComplianceServiceApplication.java
    api/ComplianceController.java
    service/ComplianceService.java
  src/main/resources/
    application.yml
```

12.2 Main Class

```
package com.digibank.compliance;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ComplianceServiceApplication {
    public static void main (String[] args) {
SpringApplication.run(ComplianceServiceApplication.class ,
args);
}
}
```

12.3 Business Service Compliance Service

```
package com.digibank.compliance.service;

import org.springframework.stereotype.Service;

@Service
public class ComplianceService {

    public boolean validateTransactionAmount (Double amount) {
        return amount != null && amount > 0 && amount <= 10000 ;
    }
}
```

```
}  
}
```

12.4 REST Controller ComplianceController

```
package com.digibank.compliance.api;  
  
import com.digibank.compliance.service.ComplianceService;  
import org.springframework.web.bind.annotation.*;  
  
import java.util.Map;  
  
@RestController  
@RequestMapping ( "/api/compliance" )  
public class ComplianceController {  
  
    private final ComplianceService service ;  
  
    public ComplianceController (ComplianceService service) {  
        this.service = service ;  
    }  
  
    @GetMapping ( "/validate/{amount}" )  
    public Map<String, Boolean> validate ( @PathVariable Double  
amount) {  
        return Map. of ( "valid" , service  
.validateTransactionAmount(amount)) ;  
    }  
}
```

```
}  
}
```

12.5 Configuring application.yml

```
server :  
  port : 8084  
  
spring :  
  application :  
    name : compliance-service
```

13. Notification Service

The notification service illustrates a secondary process triggered by a business event. Without this service, Digi Bank would not clearly demonstrate the event-driven nature of the process or the separation between the primary process and the side effect.

13.1 Service Structure

```
notification-service/  
  src/main/java/com/digibank/notification/  
    NotificationServiceApplication.java  
    api/NotificationController.java  
    service/NotificationService.java  
    model/NotificationRequest.java  
  src/main/resources/  
    application.yml
```

13.2 Main Class

```
package com.digibank.notification;  
  
import org.springframework.boot.SpringApplication;  
import  
org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class NotificationServiceApplication {  
    public static void main (String[] args) {  
SpringApplication.run(NotificationServiceApplication.class ,  
args) ;  
}  
}
```

13.3 NotificationRequest Query Template

```
package com.digibank.notification.model;

public class NotificationRequest {

    private String recipient ;

    private String message ;

    public NotificationRequest () {

    }

    public NotificationRequest (String recipient, String message)
    {

        this.recipient = recipient ;

        this.message = message ;

    }

    public String getRecipient () {

        return recipient ;

    }

    public void setRecipient (String recipient) {

        this.recipient = recipient ;

    }

    public String getMessage () {

        return message ;

    }

    public void setMessage (String message) {
```

```
        this.message = message ;  
    }  
}
```

13.4 Business Service NotificationService

```
package com.digibank.notification.service;  
  
import com.digibank.notification.model.NotificationRequest;  
import org.springframework.stereotype.Service;  
  
@Service  
public class NotificationService {  
  
    public String sendNotification (NotificationRequest request)  
    {  
        return "Notification sent to " + request.getRecipient() +  
": " + request.getMessage() ;  
    }  
}
```

13.5 REST Controller NotificationController

```
package com.digibank.notification.api;
```



```

import com.digibank.notification.model.NotificationRequest;
import com.digibank.notification.service.NotificationService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping ( "/api/notifications" )
public class NotificationController {

    private final NotificationService service ;

    public NotificationController (NotificationService service) {
        this.service = service ;
    }

    @PostMapping
    public String notify ( @RequestBody NotificationRequest
request) {

        return service .sendNotification(request) ;
    }
}

```

13.6 Configuring application.yml

```

server :

    port : 8085

spring :

```

```
application :  
  name : notification-service
```

Part II: Design Patterns, Containerization, and CI/CD

This section presents the essential patterns to demonstrate in Digi Bank: API Gateway, Service Discovery, Circuit Breaker, Saga, and CQRS. For each, we specify the concrete problem, the solution provided, how the pattern solves the problem, and then a simple implementation with the main configurations.

14. API Gateway

Without the **API Gateway**, the client must know and call each service directly. In Digi Bank, this means multiple ports, URLs, and entry points to maintain, complicating testing, security, and system evolution. The problem is even more pronounced when the frontend, Postman, or another consumer needs to communicate with customer-service, account-service, transaction-service, and compliance-service separately.

The API Gateway provides a single point of entry. It receives all external requests, routes them to the correct service, and can apply cross-functional processes such as authentication, logging, and rate limiting. In Digi Bank, it simplifies access to the architecture and hides internal complexity.

14.1 Job Role

The API Gateway receives requests from the client and then redirects them to the relevant service. It can also apply cross-functional filters, such as authentication, rate limiting, logging, or request transformation. This avoids duplicating these processes in each business service.

14.2 Service Structure

```
api-gateway/  
src/main/java/com/digibank/gateway/  
    ApiGatewayApplication.java  
src/main/resources/  
    application.yml
```

14.3 Maven Dependency

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" >

<modelVersion> 4.0.0 </modelVersion>

<parent>

<groupId> com.digibank </groupId>

<artifactId> digibank-microservices-parent </artifactId>

<version> 1.0.0 </version>

</parent>

<artifactId> api-gateway </artifactId>

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-starter-gateway </artifactId>

</dependency>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-starter-actuator </artifactId>

</dependency>
```

```

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-starter-loadbalancer </artifactId>

</dependency>

</dependencies>

<dependencyManagement>

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-dependencies </artifactId>

<version> ${spring-cloud.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>

</project>

```

14.4 Main Class

```

package com.digibank.gateway;

```

```

import org.springframework.boot.SpringApplication;

import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class ApiGatewayApplication {

    public static void main (String[] args) {
SpringApplication.run(ApiGatewayApplication. class , args);
    }

}

```

14.5 Configuring application.yml

```

server :

    port : 8080


spring :

    application :

        name : api-gateway


cloud :

    gateway :

        routes :

```

```
- ID : customer-service

    URI : http://localhost:8081

    predicates :

- Path=/api/customers/**

- id : account-service

    URI : http://localhost:8082

    predicates :

- Path=/api/accounts/**

- id : transaction-service

    URI : http://localhost:8083

    predicates :

- Path=/api/transactions/**

- ID : compliance-service

    URI : http://localhost:8084

    predicates :

- Path=/api/compliance/**

- id : notification-service

    URI : http://localhost:8085

    predicates :

- Path=/api/notifications/**
```

```
management :  
  
  endpoints :  
  
    web :  
  
      exposure :  
  
        include : gateway, routes, health, info
```

14.6 Expected result

How does the pattern solve the problem?

The client no longer directly calls multiple services; it uses a single URL. The services remain hidden, ports can change without impacting the consumer, and common rules are centralized.

The client must be able to use a single URL, for example `http://localhost:8080`, and then call the various services without knowing their internal ports. This greatly simplifies testing and prepares the architecture for centralized authentication and monitoring.

15. Discovery Service

Without **Service Discovery**, each service must know the exact address of the others. In Digi Bank, transaction-service could call compliance-service via a hard-coded URL. If the service changes ports or is restarted in a different container, the call becomes unreliable. This is a classic breaking point when moving to a distributed architecture.

Service **Discovery** solves this problem by introducing a registry where services register dynamically. Calls are made by logical name rather than physical address. This makes the architecture more flexible and more compatible with Docker deployments.

15.1 Job Role

In Digi Bank, **the transaction service** can call **the compliance service** , and then **the notification service** . With **Service Discovery** , it doesn't need to encode static URLs. It simply requests the service by name, and the registry provides the active address. This makes the system more flexible and better suited to container deployments.

15.2 Service Structure

```
discovery-server/  
  src/main/java/com/digibank/discovery/  
    DiscoveryServerApplication.java  
  src/main/resources/  
    application.yml
```

15.3 Maven Dependency

```
<project xmlns="http://maven.apache.org/POM/4.0.0" >  
<modelVersion> 4.0.0 </modelVersion>  
  
<parent>  
<groupId> com.digibank </groupId>  
<artifactId> digibank-microservices-parent </artifactId>  
<version> 1.0.0 </version>
```

```
</parent>

<artifactId> config-server </artifactId>

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId>         spring-cloud-starter-netflix-eureka-server
</artifactId>

</dependency>

</dependencies>

<dependencyManagement>

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-dependencies </artifactId>

<version> ${spring-cloud.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>

</project>
```

15.4 Main Class

```
package com.digibank.discovery;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.
SpringBootApplication ;

import org.springframework.cloud.netflix.eureka.server.
EnableEurekaServer ;

@EnableEurekaServer
@SpringBootApplication
public class DiscoveryServerApplication {

    public static void main (String[] args) {

SpringApplication.    run    (DiscoveryServerApplication.    class    ,
args) ;

    }

}
```

15.5 Configuring application.yml

```
server :

port : 8761
```

```
spring :  
  
  application :  
  
    name : discovery-server  
  
eureka :  
  
  customer :  
  
    register-with-eureka : false  
  
    fetch-registry : false
```

15.6 Client-side configuration

Each client microservice that includes the Gateway API must include the **Eureka client starter** and register with the registry. To do this, you need to add this configuration to the microservice's **pom.xml** file.

```
<dependencies>  
  
<dependency>  
  
<groupId> org.springframework.cloud </groupId>  
  
<artifactId>          spring-cloud-starter-netflix-eureka-client  
</artifactId>  
  
</dependency>  
  
</dependencies>  
  
<dependencyManagement>
```

```
<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-dependencies </artifactId>

<version> ${spring-cloud.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>
```

Also add this configuration to the **application.yml** file.

```
eureka :
  customer :
    service-url :
      defaultZone : http://localhost:8761/eureka
    register-with-eureka : true
    fetch-registry : true
```

15.7 Expected result

How does the pattern solve the problem?

Services no longer rely on fixed URLs. They are found by name, which simplifies local environments, containers, and scaling.

The discovery server must start first, then the other services must register with it automatically. The student must be able to verify in the Eureka interface that the services are visible and active.

The screenshot shows the Spring Eureka web interface in a browser window at localhost:8761. The interface has a dark header with the Spring Eureka logo and navigation links. Below the header, the 'État du système' (System Status) section displays a table with system information. To the right, a sidebar shows additional status details. Below this, the 'Répliques DS' (Datacenter Replicas) section contains a search bar. The main section, 'Instances actuellement enregistrées auprès d'Eureka' (Instances currently registered with Eureka), features a table listing various services and their status.

Application	AMI	Zones de disponibilité	Statut
SERVICE COMPTE	n/a (1)	(1)	UP (1) - mbp-de-willy:service-compte:8082
Passerelle API	n/a (1)	(1)	UP (1) - mbp-de-willy:api-gateway:8080
SERVICE DE CONFORMITÉ	n/a (1)	(1)	UP (1) - mbp-de-willy:service-de-conformité:8084
SERVICE CLIENT	n/a (1)	(1)	UP (1) - mbp-de-willy:service-client:8081
SERVICE DE NOTIFICATION	n/a (1)	(1)	UP (1) - mbp-de-willy:service-de-notification:8085
SERVICE DE TRANSACTION	n/a (1)	(1)	UP (1) - mbp-de-willy:service-transaction:8083

16. Config Server

The Config Server centralizes service configuration files. Without this component, each service retains its settings locally, complicating maintenance when modifying a port, profile, or database configuration across multiple services simultaneously.

16.1 Job Role

In Digi Bank, several services use the same configuration conventions. The Config Server avoids duplication and allows common settings to be externalized. This is a good way to show students how to properly manage a distributed application.

16.2 Service Structure

```
config-server/  
  src/main/java/com/digibank/config/  
    ConfigServerApplication.java  
  src/main/resources/  
    application.yml
```

16.3 Maven Dependency

```
<project xmlns="http://maven.apache.org/POM/4.0.0" >  
<modelVersion> 4.0.0 </modelVersion>  
  
<parent>  
<groupId> com.digibank </groupId>  
<artifactId> digibank-microservices-parent </artifactId>  
<version> 1.0.0 </version>  
</parent>  
  
<artifactId> config-server </artifactId>
```

```

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-config-server </artifactId>

</dependency>

</dependencies>

<dependencyManagement>

<dependencies>

<dependency>

<groupId> org.springframework.cloud </groupId>

<artifactId> spring-cloud-dependencies </artifactId>

<version> ${spring-cloud.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>

</project>

```

16.4 Main Class

```
package com.digibank.config;
```



```
import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.
SpringBootApplication ;

import org.springframework.cloud.config.server.
EnableConfigServer ;

@EnableConfigServer

@SpringBootApplication

public class ConfigServerApplication {

    public static void main (String[] args) {

SpringApplication. run (ConfigServerApplication. class , args);

    }

}
```

16.5 Configuring application.yml

```
server :

    port : 8888


spring :

    application :

        name : config-server

    cloud :
```

```
configuration :  
  
  server :  
  
    git :  
  
      uri : https://github.com/your-org/digibank-config-repo  
  
      default-label : main
```

In the configuration repository, you can, for example, create an `api-gateway.yml` configuration file to store the API gateway configuration.

16.6 Example configuration for the Gateway API

In the API Gateway, you need to add the following dependency to the `pom.xml` file.

```
<dependency>  
  
<groupId> org.springframework.cloud </groupId>  
  
<artifactId> spring-cloud-starter-config </artifactId>  
  
</dependency>
```

Next, modify the `application.yml` configuration file as follows:

```
server :  
  
  port : 8080
```

```
spring :  
  
  application :  
  
    name : api-gateway  
  
  configuration :  
  
    import : "configserver:http://localhost:8888"  
  
  cloud :  
  
    configuration :  
  
      fail-fast : true
```

16.7 Expected result

Microservices must be able to start by reading their parameters from the configuration server. This allows configuration changes without recompiling the business logic.

17. Circuit Breaker (application resilience)

Without a circuit breaker, a service calling another failed service continues to wait, fail, and retry. In Digi Bank, if a transaction service depends on a compliance service and the latter fails, requests become blocked and can create a cascading effect throughout the entire chain. The system becomes slow, unstable, and difficult to operate.

The Circuit Breaker temporarily cuts off calls to the down or slow service. It protects the system, prevents resource depletion, and allows for the implementation of a backup strategy. In a banking context, this is essential for maintaining overall responsiveness.

17.1 Maven Dependency

```
<dependency>  
  
<groupId> io.github.resilience4j </groupId>  
  
<artifactId> resilience4j-spring-boot3 </artifactId>  
  
</dependency>
```

17.2 Implementing ComplianceClient with CircuitBreaker

```
package com.digibank.transaction.client;  
  
import  
io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;  
  
import org.springframework.stereotype.Component;  
  
import org.springframework.beans.factory.annotation.Value;  
  
import org.springframework.web.client.RestTemplate;
```

```

@Component
public class ComplianceClient {

    private final RestTemplate restTemplate;

    public ComplianceClient(RestTemplate restTemplate) {
        this .restTemplate = restTemplate;
    }

    @CircuitBreaker(name = "complianceService" , fallbackMethod =
    "fallbackValidation" )

    public boolean validateAmount(Double amount) {
        Boolean result = restTemplate.getForObject(
            "http://localhost:8084/api/compliance/validate/"
            + amount,
            Boolean.class
        );

        return Boolean.TRUE.equals(result);
    }

    public boolean fallbackValidation(Double amount, Throwable
    ex) {
        return false ;
    }
}

```

17.3 Configuration Update

Add this configuration to **application.yml**

```
resilience4j :  
  circuitbreaker :  
    instances :  
      complianceService :  
        slidingWindowSize : 10  
        failureRateThreshold : 50  
        waitDurationInOpenState : 10s  
        permittedNumberOfCallsInHalfOpenState : 3
```

17.4 Expected Result

How does the pattern solve the problem?

The service does not persist in calling a component that is already failed. It switches to degraded mode more quickly and protects the entire architecture.

18. SAGA (Distributed Operations)

Without **SAGA**, a distributed transaction may only partially succeed. In Digi Bank, a transfer can debit an account, then fail to record the transaction or send a notification. This results in an inconsistent state, which is unacceptable in a banking system.

SAGA breaks down a global operation into several coordinated local steps. Each step can have a compensatory action if a subsequent step fails. The system thus avoids relying on a single global ACID transaction between multiple services .

18.1 Implementation of a TransferSagaService transfer orchestrator

```
package com.digibank.transaction.service;

import com.digibank.transaction.client.AccountClient;
import com.digibank.transaction.client.ComplianceClient;
import com.digibank.transaction.client.NotificationClient;
import org.springframework.stereotype. Service ;

@Service
public class TransferSagaService {

    private final ComplianceClient complianceClient ;

    private final AccountClient accountClient ;

    private final NotificationClient notificationClient ;

    public TransferSagaService (
ComplianceClient complianceClient,
AccountClient accountClient,
NotificationClient notificationClient) {
```

```
this . complianceClient = complianceClient;

this . accountClient = accountClient;

this . notificationClient = notificationClient;
}

public boolean executeTransfer (Long fromAccountId, Long
toAccountId, Double amount) {

    if (! complianceClient .validateAmount(amount)) {

        return false ;
    }

    boolean debited = accountClient .debit(fromAccountId,
amount);

    if (!debited) {

        return false ;
    }

    boolean credited = accountClient .credit(toAccountId,
amount);

    if (!credited) {

        accountClient .compensateDebit(fromAccountId,
amount);

        return false ;
    }
}
```



```

        boolean notified = notificationClient
.sendTransferConfirmation(fromAccountId, toAccountId, amount);

        if (!notified) {

            accountClient .compensateDebit(fromAccountId,
amount);

            accountClient .compensateCredit(toAccountId, amount);

            return false ;
        }

        return true ;
    }
}

```

18.2 Client Account Implementation

```

package com.digibank.transaction.client;

import org.springframework.stereotype. Component ;

@Component
public class AccountClient {

    public boolean debit (Long fromAccountId, Double amount) {

        // @ TODO implement method
    }
}

```

```

        return true ;
    }

    public boolean credit (Long toAccountId, Double amount) {
        // @ TODO implement method

        return true ;
    }

    public void compensateDebit (Long fromAccountId, Double
amount) {
        // @ TODO implement method
    }

    public void compensateCredit (Long toAccountId, Double
amount) {
        // @ TODO implement method
    }
}

```

18.3 Client Notification Implementation

```

package com.digibank.transaction.client;

import org.springframework.stereotype. Component ;

```

```

@Component
public class NotificationClient {

    public boolean sendTransferConfirmation (Long fromAccountId,
Long toAccountId, Double amount) {

        // @ TODO implement method

        return true ;
    }
}

```

18.4 Expected Result

How does the pattern solve the problem?

Saga ensures that a distributed business operation remains consistent, even if a step fails. It provides controlled and compensated behavior instead of a partial state that cannot be manually repaired.

19. CQRS (Command Query Responsibility Segregation)

Without **CQRS** , Digi Bank often uses the same model for both reading and writing. This becomes cumbersome as soon as a view becomes complex, such as a customer's complete history, activity aggregates, or dashboards. While write operations need to remain simple, read operations can have very different requirements.

CQRS separates the command model from the reading model. This allows each side to be optimized for its actual use. In Digi Bank, this is useful for transaction history screens or account summary views.

19.1 Implementation of a command

```
package com.digibank.transaction.command;

public record CreateTransactionCommand(Long accountId , Double
amount , String type ) {

}
```

19.2 Implementation of the write handler

```
package com.digibank.transaction.command;

import com.digibank.transaction.model.Transaction;

import
com.digibank.transaction.repository.TransactionRepository;

import org.springframework.stereotype. Service ;

@Service

public class TransactionCommandHandler {

    private final TransactionRepository repository ;


    public TransactionCommandHandler (TransactionRepository
repository) {
```

```

        this . repository = repository;
    }

    public Transaction handle (CreateTransactionCommand command)
    {
        Transaction tx = new Transaction();
        tx.setAccountId(command.accountId());
        tx.setAmount(command.amount());
        tx.setType(command.type());

        return repository .save(tx);
    }
}

```

19.3 Implementation of a reading view

```

package com.digibank.transaction.query;

public record TransactionView(Long id , Long accountId , Double
amount , String type ) {
}

```

19.4 Implementation of a reading service

```

package com.digibank.transaction.query;

import
com.digibank.transaction.repository.TransactionRepository;

import org.springframework.stereotype. Service ;

import java.util.List;

@Service

public class TransactionQueryService {

    private final TransactionRepository repository ;

    public TransactionQueryService (TransactionRepository
repository) {

        this . repository = repository;
    }

    public List<TransactionView> getAllViews () {

        return repository.findAll ()

        .stream()

        .map(tx -> new TransactionView(tx.getId(), tx.getAccountId(),
tx.getAmount(), tx.getType()))

        .toList();
    }

```

```
}
```

19.5 Expected result

How does the pattern solve the problem?

CQRS simplifies complex reads and maintains clean writes. It makes views easier to optimize without degrading the command path.

20. Global configuration

For these patterns to work correctly in Digi Bank, several common configurations must be implemented:

- One application.yml file per service
- Eureka for the discovery
- Spring Cloud Gateway for routing
- Resilience4j for flow protections
- PostgreSQL by service
- Stable service names
- Docker Compose to launch the whole thing.

For Digi Bank, the logical order is as follows:

1. Create basic business services
2. Add Discovery Service
3. Introduce the Gateway API
4. Secure calls with Circuit Breaker
5. Modeling a Transfer Saga
6. Separating reading and writing with CQRS
7. Containerize and orchestrate the whole thing.

This order allows progress from the simplest to the most complex, without mixing different areas of focus. It also provides students with a coherent and progressive learning path.

21. Tests

Testing is essential in Digi Bank because a microservices architecture multiplies the potential points of failure. Without testing, a regression in a service can break an entire chain of calls, particularly between transaction services, compliance services, and notification services. Therefore, it's crucial to distinguish between unit tests, integration tests, and end-to-end tests.

21.1 Unit Tests

Unit tests verify the behavior of a class or an isolated service. In Digi Bank, they are used, for example, to validate the compliance rule, the logic for creating an account, or the processing of a notification.

Example for the compliance department:

```
package com.digibank.compliance.service;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api. Test ;

public class ComplianceServiceTest {

    private final ComplianceService service = new
ComplianceService();
```



```

@Test
void shouldAcceptAmountBelowLimit () {
    Assertions. assertTrue ( service.validateTransactionAmount (
5000.0 ));
}

@Test
void shouldRejectAmountAboveLimit () {
    Assertions. assertFalse ( service.validateTransactionAmount (
20000.0 ));
}
}

```

21.2 Integration Tests

Integration tests verify that a service communicates correctly with its database or another service. In Digi Bank, they are useful for checking that customer-service correctly persists customer information and that transaction-service correctly calls compliance-service.

Example :

- Start the Spring context
- Inject the repository
- To persist an entity
- Check that it has been proofread correctly.

To make them you can use **cucumber**

21.3 API Testing

API tests allow you to verify the REST endpoints exposed by each microservice. They are particularly useful for validating the gateway, routes, and business responses. A simple example is to test that ``GET /api/compliance/validate/5000`` returns a positive response. You can use **`newman` to do this.**

21.4. Expected result

The tests must demonstrate that each service responds correctly, that business rules are respected, and that errors are detected before deployment.

22. Dockerization of services

Docker allows each component to run in an isolated container. Without containerization, local installation and execution become less reproducible, especially when multiple databases, services, and dependencies need to be launched in parallel.

22.1 Why Docker in Digi Bank

Docker is useful for:

- Isolate each microservice
- Standardize the execution environment
- Replicating the same architecture across multiple machines
- Facilitate integration testing
- Prepare for a cloud or CI/CD deployment.

22.2 Dockerfile type for a Spring Boot service

Each Spring Boot service must be packaged in a standalone Docker image. Without this, the orchestration would be fragile and dependent on the local environment. The

principle is the same for all services, with only the port, the JAR name, and sometimes the environment variables changing.

22.3 Dockerfile for customer-service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / customer-service-1.0.0.jar app.jar

EXPOSE 8081

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.4 Dockerfile for account-service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / account-service-1.0.0.jar app.jar

EXPOSE 8082

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.5 Dockerfile for transaction-service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / transaction-service-1.0.0.jar app.jar

EXPOSE 8083

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.6 Dockerfile for compliance-service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / compliance-service-1.0.0.jar app.jar

EXPOSE 8084

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.7 Dockerfile for notification-service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / notification-service-1.0.0.jar app.jar

EXPOSE 8085
```

```
ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.8 Dockerfile for the API gateway

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / api-gateway-1.0.0.jar app.jar

EXPOSE 8080

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.9 Dockerfile for the discovery service

```
FROM eclipse-temurin:17-jdk

WORKDIR / app

COPY target / discovery-server-1.0.0.jar app.jar

EXPOSE 8761

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.10 Dockerfile for the server configuration

```
FROM eclipse-temurin:17-jdk

WORKDIR / app
```

```
COPY target / config-server-1.0.0.jar app.jar

EXPOSE 8888

ENTRYPOINT [ "java" , "-jar" , "/app/app.jar" ]
```

22.11 Expected result

Each service must be able to be built as an independent Docker image. The student must be able to run one of the services locally without installing anything other than Docker and Java.

23. Orchestration with Docker Compose

Docker Compose allows you to start the entire Digi Bank system with a single command. Without orchestration, you would have to manually launch each database, service, and infrastructure component, which would be tedious and unrealistic for a distributed environment.

Docker Compose allows you to launch the entire Digi Bank ecosystem all at once. This is essential for a microservices architecture, because the system is only valuable if the services communicate properly with each other.

18.1 docker-compose.yml file

```
version : "3.9"

services :

  discovery-server :
```

```
build : ./discovery-server

ports :
- "8761:8761"

API Gateway :

build : ./api-gateway

ports :
- "8080:8080"

depends_on :
- discovery-server

customer-db :

image : postgres:16

container_name : customer-db

environment :

    POSTGRES_DB : digibank_customer_db

    POSTGRES_USER : digibankuser

    POSTGRES_PASSWORD : digibankpwd

ports :
- "5433:5432"

volumes :
```

```
- customer_data:/var/lib/postgresql/data

account-db :

  image : postgres:16

  container_name : account-db

  environment :

    POSTGRES_DB : digibank_account_db

    POSTGRES_USER : digibankuser

    POSTGRES_PASSWORD : digibankpwd

  ports :

- "5434:5432"

  volumes :

- account_data:/var/lib/postgresql/data

transaction-db :

  image : postgres:16

  container_name : transaction-db

  environment :

    POSTGRES_DB : digibank_transaction_db

    POSTGRES_USER : digibankuser

    POSTGRES_PASSWORD : digibankpwd
```



```
ports :  
- "5435:5432"  
  
volumes :  
- transaction_data:/var/lib/postgresql/data  
  
compliance-db :  
  
  image : postgres:16  
  
  container_name : compliance-db  
  
  environment :  
  
    POSTGRES_DB : digibank_compliance_db  
  
    POSTGRES_USER : digibankuser  
  
    POSTGRES_PASSWORD : digibankpwd  
  
  ports :  
- "5436:5432"  
  
  volumes :  
- compliance_data:/var/lib/postgresql/data  
  
notification-db :  
  
  image : postgres:16  
  
  container_name : notification-db  
  
  environment :
```

```
    POSTGRES_DB : digibank_notification_db

    POSTGRES_USER : digibankuser

    POSTGRES_PASSWORD : digibankpwd

    ports :
- "5437:5432"

    volumes :
- notification_data:/var/lib/postgresql/data

customer service :

    build : ./customer-service

    ports :
- "8081:8081"

    depends_on :
- customer-db
- discovery-server

account-service :

    build : ./account-service

    ports :
- "8082:8082"

    depends_on :
```

```
- account-db

- discovery-server

transaction-service :

  build : ./transaction-service

  ports :
- "8083:8083"

  depends_on :
- transaction-db
- discovery-server

compliance service :

  build : ./compliance-service

  ports :
- "8084:8084"

  depends_on :
- compliance-db

notification-service :

  build : ./notification-service

  ports :
```

```
- "8085:8085"

  depends_on :
- notification-db

volumes :

  customer_data :

  account_data :

  transaction_data :

  compliance_data :

  notification_data :
```

18.3 Launch Commands

```
docker compound up --build
```

18.4 Expected Result

The entire Digi Bank system must start up consistently. The Eureka server must be accessible, the API Gateway must be responsive, and the business services must be reachable via their respective endpoints.

24. Github Actions Pipeline (CI/CD)

CI/CD automation is useful for ensuring that every code change remains compilable and testable. In Digi Bank, it allows for the rapid detection of regressions introduced into a service or the global configuration. [Attachment]

24.1 Pipeline Objective

The pipeline must:

- Compile the projects
- Run the tests
- To produce the artifacts
- Report any build failures
- Optionally prepare the Docker image build.

24.2 Workflow example `.github/workflows/ci.yml`

```
Name : DigiBank CI

on :

  push :

    branches : [ "main" ]

  pull_request :

jobs :

  build :

    runs-on : ubuntu-latest
```

```
steps :  
  
- name : Checkout code  
  uses : actions/checkout@v4  
  
- name : JDK 17 Setup  
  uses : actions/setup-java@v4  
  with :  
    distribution : temurin  
    Java version : "17"  
  
- name : Maven packages cache  
  uses : actions/cache@v4  
  with :  
    path : ~/.m2/repository  
    key : ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}  
  }}  
  
  restore-keys : |  
${{ runner.os }}-m2  
  
- name : Build and test  
  run : mvn clean test
```



24.3 Expected Result

With each push, the project must be compiled and tested automatically. The student must understand that in microservices, automation becomes almost indispensable for maintaining a manageable architecture.

Part III: Tasks to be completed

25. Work requested

Students must complete the following tasks:

1. Create the Spring Boot services for the project.
2. Configure PostgreSQL databases by service.
3. Set up Eureka for discovery.
4. Set up the configuration server.
5. Implement the API Gateway.
6. Add the Circuit Breaker to inter-service calls.
7. Building a Transfer Saga.
8. Separating reading and writing with CQRS.
9. Containerize services with Docker.
10. Orchestrate everything with Docker Compose.
11. Automate build and deploy with GitHub Actions.
12. Verify the endpoints and business behaviors.

13. Correct the errors until a stable set is obtained.

26. Expected deliverables

Students must submit a complete and coherent project, along with the necessary configuration and documentation to allow for the reproduction of the Digi Bank architecture. The goal is not simply to provide code, but to demonstrate an approach to architecture, implementation, and validation.

The expected deliverables are as follows:

- The complete digibank-microservices deposit
- All Maven projects created
- The pom.xml files for each service
- The application.yml or application.properties files
- The Dockerfile for each component
- The docker-compose.yml file
- GitHub Actions Workflows
- The Java classes of business services
- REST controllers
- Repositories, services, clients, and configurations
- Unit and integration testing
- The SQL scripts for creating the databases
- A short note explaining the order of execution and the technical choices.

27. Evaluation criteria

The evaluation is based on technical consistency, the quality of the architecture's structure, and its ability to function as a distributed system. It's not simply about verifying that a service starts, but about measuring the robustness of the overall design. The criteria used are as follows:

- Consistency of the microservices architecture
- Relevance of the job classification
- Correctness of Spring Boot configurations
- Quality of interdepartmental calls
- Successful integration of the Gateway API
- How the Discovery Service Works
- Implementation of resilience mechanisms
- Quality of Saga and CQRS implementation
- Validity of Dockerfiles and docker-compose.yml
- GitHub Actions pipeline functioning correctly
- Quality and relevance of the tests
- Clarity of the documentation provided.

28. Proposed scale

- Environment preparation and configuration: 10%
- Project structure and coherence: 15%
- Implementation of business services: 25%
- API Gateway, Discovery and Config Server: 15%
- Microservices patterns: 15%
- Containerization and orchestration: 10%
- CI/CD testing and automation: 10%.

29. Expected results

At the end of this workshop, students should have a first executable version of Digi Bank based on a coherent microservices architecture. The application must include a unified entry point, autonomous business services, dynamic service discovery, centralized configuration, resilience mechanisms, and functional Docker orchestration.



The system must also be documented, testable, and automatable. The student must be able to explain the role of each pattern, launch the services, verify the endpoints, and justify the technical choices adopted for the project.